3X-UI Manager — User Manual

English · Русский

App version: 0.10.8. This manual is based on and current for that version.

Get the app: F-Droid · GitHub Releases · source code — see section 2 for the differences between builds.

> A full guide to **3X-UI Manager**, the Android app that manages a 3x-ui panel from your phone: dashboard, inbounds, clients, nodes and Xray settings.
>
> Names and labels match the app's interface. The words *inbound / outbound* are left as they are — that is what the panel itself calls them.

Contents

1. What this is and what you need

1.1. What the app does

3X-UI Manager is a **management client** for the 3x-ui panel. It connects to a panel that is already running and gives you from your phone roughly what the web interface gives you:

- a dashboard with server and Xray state, load charts and the list of clients online;
- full work with **inbounds** — create, edit, enable/disable;
- work with **clients**: quotas, expiry, search and filters, bulk actions, handing out links and QR codes, IP log;
- management of **nodes** (the master/node multi-panel setup), including mTLS;
- editing the **Xray** configuration: outbounds, routing, DNS, logs, and when needed the whole config as text;
- **administration** of the panel itself: account, API tokens, mail settings, restart;
- **backups** of the panel database;
- **local notifications** about problems: expired clients, exhausted quotas, an unreachable port, a stopped Xray, a node that dropped off.

1.2. Requirements

Item	Requirement
Android	7.0 or newer (API 24)
3x-ui panel	**v3.3.0 or newer** — that is the version from which the panel accepts API-token auth. Some capabilities arrived later and are unavailable on an old panel
Access	the phone must be able to reach the panel's address (directly, over a VPN, or through a reverse proxy)

Keeping the panel on its latest version is recommended — that way every section of the app is available, along with the panel's own fixes. What is tied to which version is listed in section 17. When the panel is older than a given feature needs, the app does not hide the section: it says inside it that your panel version doesn't support this yet and suggests updating.

1.3. What the app does not do

This is **not a VPN client**. It does not connect your phone to your proxies and does not route traffic through them — it only manages the panel. To use the connection itself you need an ordinary client (v2rayNG, Hiddify, Streisand and so on); the app helps you hand it a link or a QR code.

The app also does not install or set up the panel on a server — the panel has to be deployed already.

2. Installing and updating

2.1. Where to get it

Source	Link	Notes
F-Droid	https://f-droid.org/packages/net.yukh.xui	recommended: updates arrive through the catalog
GitHub Releases	https://github.com/yukh975/3X-UI-Manager/releases	download the APK by hand
Obtainium	add the repository `https://github.com/yukh975/3X-UI-Manager`	auto-updates straight from GitHub

A GitHub release carries **two** APKs. The ordinary one is named `3x-ui-manager-<version>.apk` — that is the one to take. The `fdroid.apk` file is the reference binary for F-Droid's build server; there is no reason to install it by hand.

2.2. How the builds differ

The app is built in two variants, and the only difference is the update mechanism:

- the **standard build** (GitHub, Obtainium) can check for updates itself and offer to install them;
- the **F-Droid build** cannot, because the catalog updates apps itself and F-Droid's rules forbid an app from downloading and installing its own APK. It doesn't even carry the corresponding system permission.

Both builds are signed with the same key, so you can install one over the other without uninstalling and without losing data.

2.3. How updates arrive

The standard build checks for a newer version at startup and shows a dialog with the list of changes. The text follows the interface language: with Russian selected you get the Russian changelog, not the English release body. You can also check manually under **About → Check for updates**.

In the F-Droid build, that button is replaced by a line explaining that updates come through the catalog.

---

3. Connecting to a panel

The app connects **with an API token**. The administrator's login and password are not used — a token is both safer and doesn't break when the password changes.

3.1. Where to get the API token

1. Open the panel in a browser.
2. Go to **Settings** → **Security** → **API Token**.
3. Create a token and copy its value.

> **A 3x-ui token is full administrator access.** Whoever holds it can do everything you can do through the web panel. Treat it like a password; if the phone is lost, delete the token in the panel and the app loses access.

3.2. The connection form

Field	What to enter
Panel URL	the address including scheme and port, e.g. `https://panel.example.com:2053`. If your administrator set a secret path (`webBasePath`), include it
API token	the value copied from the panel. "Show token" lets you check that it pasted in full
Allow self-signed TLS	turns off certificate verification. Enable it only if the panel uses a self-signed certificate — and only for your own panel
Subscription base URL (optional)	rarely needed. The app reads the subscription address from the panel settings itself; set it here only when the public address differs — for example a panel behind a reverse proxy: `https://host:2096/sub/`

Once the connection succeeds the profile is saved, and the app reconnects on its own next time.

3.3. When the connection fails

- **An "unexpected response" error** usually means the app received HTML rather than JSON. That happens when `webBasePath` is missing from the address, or a reverse proxy answers with its own page.
- **A certificate error** — turn on "Allow self-signed TLS".
- **401, or "your API token is no longer valid"** — the token was deleted or disabled in the panel; create a new one.
- **Nothing answers** — check that the panel's address is reachable from the phone (mobile network, VPN, source-IP restrictions on the server).

---

4. Several panels

The app stores any number of panels and switches between them.

The ⇄ button in the top bar opens the list of saved connections, where you can:

- **switch** to another panel — every screen reloads against it;
- **add a panel** — an empty connection form opens;
- **sign out** of one panel — the app forgets its address and token.

The active panel is named in the title bar, so mixing up servers is hard. Signing out of the last panel returns the app to its fresh-install state — which also clears the app-lock passcode.

---

5. App lock

Since the app holds tokens with full access to your servers, you can lock it behind a passcode. Under **Settings → App lock**:

- **Set passcode** — 4 to 8 digits;
- **Biometric unlock** — fingerprint or face, if the device supports it;
- **Remove passcode**.

How it behaves in practice:

- at startup the passcode is asked for when one is set and a saved session was restored automatically; right after you typed a token in by hand, you are not asked again;
- sending the app to the background re-locks it, but with a **30-second grace period** — a quick switch to another app (to copy an address or read a code) and back doesn't prompt for the passcode;
- unlocking works with either the passcode or biometrics.

---

6. Dashboard

The first tab. It refreshes on its own; pull the list down to refresh at once.

6.1. Xray state

The card shows whether Xray is running, and its version. The buttons:

- **Restart** — applies configuration changes; it briefly drops every active client connection, which the app warns you about;
- **Stop** / **Start**.

After you press one, the app holds the expected state for a moment so a lagging poll doesn't flip the button back and forth.

6.2. Server metrics

- **CPU**, **Memory**, **Disk** — as percentages;
- **Net ↑ / ↓ per s** — current rate;
- **Connections** — TCP and UDP counts;
- **Load 1·5·15m** — system load average.

6.3. History charts

The metric blocks are tappable and open a chart over a period. The interval is switchable (the default is the "live" two-minute bucket); the panel keeps system metrics history for roughly a week. The data comes from the panel — the app stores nothing of its own.

6.4. Clients online

The online counter is tappable and opens the list. If nodes are attached to the panel, the list is grouped by server, so you can see who is on the main server and who is on a node.

6.5. Traffic this month

Total traffic across all inbounds for the current period, again broken down by server. It is summed from the inbound counters; if some inbound is not on a monthly reset, the app flags that — otherwise the sum would silently mix monthly and all-time counters.

6.6. Panel version and geo databases

A separate card shows the panel version and offers an **update** when one is available. Next to it is the **geo database** update button (geoip / geosite): the panel re-downloads them and restarts Xray.

---

7. Inbounds

7.1. The list

Each inbound is a card: remark, protocol, port, traffic counters, expiry and the live ↑/↓ rate while the screen is open. The switch enables and disables the inbound; if the panel rejects the operation, the app puts the switch back and shows the error.

7.2. Editor: common fields

- **Remark** — the name you recognise it by;
- **Protocol** — VLESS, VMess, Trojan, Shadowsocks, WireGuard, MTProto and the others the panel supports;
- **Port** and **listen address**;
- **Traffic quota** and **expiry** for the whole inbound;
- **Periodic traffic reset** — including the day of the month, if the panel supports it (3.6.0);
- **Node** — which server to deploy the inbound on, when nodes are attached.

7.3. Transport and security

Transport
(`streamSettings`) is edited through structured fields: the transport type (TCP/raw, WebSocket, gRPC, HTTPUpgrade, XHTTP, KCP and so on), its parameters, and TLS or REALITY with their fields — certificates, SNI, fingerprint, short ID.

An important detail: the app edits the transport as a live JSON object, so **keys it doesn't know about are not lost**. Editing an inbound that was created in the web panel with exotic settings will not wipe them.

7.4. Sniffing, fallbacks and raw JSON

- **Sniffing** — destination detection from traffic, with its list of types;
- **Fallbacks** — forwarding by SNI/path to other ports;
- **Inbound JSON (advanced)** — protocol settings that aren't surfaced as form fields are available as raw JSON. Clients are kept separately and are not overwritten: they are managed on the Clients tab.

7.5. Port reachability monitoring

The editor of a saved inbound has a **"Monitor reachability"** switch. It belongs to alerts: the port and remark are remembered **on the phone itself**, and the background probe then knocks on that port directly without asking the panel. It works that way because the panel is often firewalled off from the phone, which makes its reachability a poor health signal.

8. Clients

8.1. List, search and filters

The list shows, for each client: an online dot, email/name, ↑/↓ traffic against the quota, live rate, expiry and when they were last seen.

Search covers more than the email — also the comment, sub ID, UUID, password, the auth field and the Telegram ID.

Filters (the button with a count badge) mirror the web panel's:

- by status — active, disabled, expired, quota depleted;
- by group.

Within one section the conditions are OR'ed, and the sections narrow the selection independently — the same as in the panel.

8.2. Creating and editing

The main fields: **email/name**, **traffic quota** (in GB), **expiry**, **IP limit**, **group**, **comment**, **Telegram ID**, **subId**, and the binding to one or more inbounds.

The form adapts to the protocol of the selected inbound:

- for **MTProto** a FakeTLS secret (with a regenerate button) and an ad-tag appear;
- for **WireGuard** — the peer's allowed IPs;
- for VLESS — the **flow** field.

Identifiers (UUID, password, subId) are minted by the app on creation in the same format the web panel uses.

8.3. Handing a configuration to a client

Tapping a client opens the share sheet:

- **Subscription** — a single link from which a client app fetches every configuration and keeps them updated. A QR code is shown, with copy and share buttons;
- **Connections** — the individual per-server links; each expands to its own QR;
- **"As the subscriber sees it"** — the live subscription status (online, used against the quota) exactly as the client's own app receives it. Requires panel 3.6.0; if the subscription server isn't reachable from the phone, the block is simply omitted.

Edit, **Delete** and **IP log** are available from here as well.

8.4. Bulk actions

A long press turns on selection mode. With the selected clients you can:

- **enable** or **disable** them;

- **adjust** — add or subtract days and gigabytes, set the flow;
- **delete** them.

8.5. IP log

Shows which addresses the client connected from and when, and with multi-panel, through which node. The log can be cleared.

8.6. Export, import and cleanup

- **Export** all clients as JSON — useful as a backup or for moving them;
- **Import** from the same JSON; the panel skips clients whose email already exists;
- **Delete unbound clients** — remove clients not attached to any inbound.

---

9. Nodes (multi-panel)

This section matters if you run a master panel with nodes: several servers you manage from one place.

9.1. List and status

Each node shows its name, address, state, the panel version running on it and counters. The switch enables and disables the node. Pull the list down to refresh.

9.2. Adding a node

The fields: name, scheme (http/https), address, port, base path, the **node's API token**, TLS verification mode and the **connection outbound** (picked from the outbound tags of the Xray config).

Since panel 3.6.0 the node token is **not returned back** — the panel only reports whether one is set. So when editing, an empty token field means "keep the current one", and entering a value replaces it.

9.3. Mutual TLS (mTLS)

A separate page (panel 3.4.0 and newer). It covers two things:

- **copy this panel's CA**, to register it as trusted on a node;
- **set the CA** whose client certificates this panel trusts when it acts as a node.

With mTLS a node needs no API token.

9.4. Updating the panel on a node

A node's panel can be updated straight from the app. The node downloads the new build and restarts, which takes noticeably longer than a single list refresh — so the app keeps polling the node until its reported version actually changes, and only then shows the result.

---

10. Xray: outbounds, routing, DNS

Everything to do with the Xray configuration sits in the ⋮ menu in the top bar. The sections edit different parts of one and the same config, and each of them saves the config as a whole — the neighbouring parts you didn't touch are preserved.

A general rule: changes are written into the panel's configuration but reach the running core **after Xray is restarted**. The app reminds you; the restart button is on the dashboard.

10.1. Outbounds

A list of outbounds with their tag, protocol and address. The first one is the default route and is labelled as such. Available actions:

- **create and edit** an outbound: protocol, address, port, credentials, transport, plus **Target Strategy** (from `AsIs` to `ForceIPv4`) and "Send through";
- **reorder** — with the up/down arrows; the order decides which one becomes the default route;
- **delete**;
- **import from a `vless://` link**;
- **import and export** the whole list as JSON.

10.2. Testing outbounds

Every outbound has a **Test** button with a mode selector:

Mode	What it measures
TCP	reachability and delay at the connection level
HTTP	a full request through the proxy; also reports the egress IP and country
Real delay	the full time including tunnel establishment — closest to the real experience

For HTTP and Real, the delay is joined by the IP, the country flag and a **WARP** marker when the egress goes through it. The URL used for the probe is set on the panel side.

10.3. Outbound subscriptions

The panel can fetch someone else's server list from a subscription URL on a timer and merge it into the config as outbounds. The section opens from the ⋮ menu on the Outbounds screen.

Each subscription is configured with:

Field	Meaning
Subscription URL	where the panel takes the list from
Remark	how you recognise it
Tag prefix	e.g. `hk–`, to tell its servers apart in routing

Field	Meaning
Update interval	how often the panel re-reads the URL
Enabled	switch it off temporarily without deleting
Before manual outbounds	put its servers above yours — then one of them can become the default route
Allow private address	permit a URL pointing at a local/LAN address; off by default for safety
Allow insecure TLS	for a source with a self-signed certificate

Actions: **preview** (the panel fetches and parses the URL, showing how many servers were found and with which tags — before you save), **refresh now**, **refresh all**, reorder and delete.

Servers that came from a subscription appear on the Outbounds screen as a separate block. They can be **tested** and used in routing rules and balancers just like ordinary ones, but they **cannot be edited directly**: the panel regenerates them on every subscription refresh, so any edit would be overwritten. Change them through the subscription itself.

10.4. Routing and balancers

The **Routing** section holds:

- **rules** — by domain, IP, port, protocol, inbound; each rule points at an outbound or a balancer. Rules can be enabled and disabled, reordered, imported and exported;
- **balancers** — groups of outbounds with a selection strategy (`random` , `roundRobin` , `leastLoad` , `leastPing`);
- **Default Outbound** — which outbound handles traffic that matched no rule.

The internal stats rule (`api`) is protected from being disabled — without it traffic accounting breaks.

10.5. Route test

This tool answers "where would this address go". Enter a domain or IP, optionally a port and network type, **pick an inbound** — and you get the resulting outbound, the balancer chain if there is one, or a "default outbound" verdict.

Choosing the inbound is essential rather than optional: nearly all routing rules key on the inbound, and without it the router cannot decide. The test works for local inbounds; the master panel has no routing rules for node inbounds, so the result isn't meaningful for them.

10.6. DNS

An editor for the DNS section: servers (plain and conditional), the query strategy, and static hosts entries.

10.7. General settings and logs

The **General / Logs** section: log level, log access parameters and the other general core settings.

10.8. The full Xray config

If a field you need isn't in any form, the configuration can be edited as text — the whole thing, the same JSON as on the panel's Xray Configuration page. The outbound test URL is set next to it. Saving requires an Xray restart.

---

11. Panel administration

The **Panel admin** section (the ⋮ menu) gathers the panel's own settings.

- **Admin account** — change the login and password. The current ones must be entered. After the change your API token keeps working, but other browser sessions are signed out.
- **API tokens** — the panel's token list: enable, disable, delete, create. A new token's value is shown **once** — copy it right away. Deleting a token immediately cuts off every app using it.
- **Subscription** — the **announcement** text (`subAnnounce`) shown as a banner on the subscription info page.
- **Email (SMTP)** — the sender address and name for the panel's mail. Leaving the address empty falls back to the SMTP username. This matters when the relay's login is not an email address: without a valid `From`, strict receivers such as Gmail reject the message. Requires panel 3.6.0.
- **Notifications** — the **outbound-down threshold**: how many consecutive failed probes must happen before the panel sends an alert about an unreachable outbound. `1` means the old behaviour, alerting on the first failure. It helps against a flood of false alarms on a flaky link. Requires panel 3.6.0.
- **Restart panel** — restarts the panel service itself; the app's connection drops for a few seconds.

12. Backups

The **Backup / restore** section.

- **Download** — the app fetches the panel database and asks where to save the file. The panel picks the name: `x-ui.db` for SQLite or `x-ui.dump` for PostgreSQL.
- **Restore** — pick a file; the panel imports it under its own engine and restarts Xray.

That database holds **everything**: panel settings, inbounds, clients and the Xray configuration. That makes a backup the quickest way to move to another server — and, for the same reason, a file to treat as a secret.

---

13. Alerts

13.1. How it works

Alerts are **entirely local**. The app itself polls the saved panels roughly every 30 minutes and raises system notifications. There is no external push service involved: neither your data nor your panel addresses go anywhere. The flip side is that an alert appears at the next check rather than the same second.

Reachability is checked **at the TCP port level**, not through the panel API. The reason is simple: the panel is often firewalled off from the phone, so its being unreachable says nothing about the health of the service. By default **port 443** is probed — the usual public entry point.

To avoid crying wolf, an unreachability alert fires only after **two consecutive failed checks**, and an authentication error (401) is not treated as unreachability.

13.2. Kinds of alert

Alert	When
Client expired / expires soon	the expiry has passed, or is closer than the configured number of days
Traffic limit almost reached	more than the configured percentage has been used
Panel unreachable	the configured port doesn't answer
Inbound unreachable	the port of an inbound marked for monitoring doesn't answer
Xray is down	the core isn't running
Node offline	a node is off the air

Notifications are split across **two system channels** — client ones and infrastructure ones. That lets you mute the client chatter in Android settings while keeping server alarms. Each notification carries the date and time it fired, so if you see it hours later you can tell whether the problem is current.

13.3. Configuration

Under **Settings** → **Panel alerts**:

- the **enable** switch (on Android 13+ the app asks for notification permission);
- **days before expiry** — how far ahead to warn (default 3);
- **traffic threshold (%)** — at what usage to warn (default 90);
- **panel port** — which port to probe for reachability (default 443).

Individual inbounds are added to monitoring from their editor — see 7.5.

14. App settings

- **Language** — **System default** (the default), English or Русский. In system mode the app follows the phone's language and switches as soon as you change it, without a restart. An explicit choice always wins over the system one.
- **Speed units** — bytes (KB/s) or bits (Kbit/s). Applies to inbound and client rates.
- **App lock** — see section 5.

15. About

A separate item in the ⋮ menu. It holds:

- the name, **version** and copyright;
- the **update check** (in the F-Droid build, a note that updates come through the catalog instead);
- the **changelog** — the full list of versions and what changed in each. It works offline: the changelog is bundled into the app, nothing is fetched for it;
- a link to the **project on GitHub**;
- the **Our projects** block — the 3x-ui manual, Currency Converter and netadm.pro.

Before you connect to a panel the section is reachable from Settings, so the version can be checked without signing in.

16. Privacy and security

Where the app connects. Only to the addresses you entered yourself: your panel and, when handing a subscription to a client, its subscription server. Plus, in the standard build, the app's own releases page to check for updates. There is no analytics, no advertising and no trackers in the app.

The F-Droid build never contacts our infrastructure at all — it doesn't even have the update check. A side effect of being honest about it: we have no idea how many people use the app from the catalog.

How your data is stored. Connection profiles (the panel address and API token) live in Android's encrypted storage: values are encrypted with AES-256-GCM, keys with AES-256-SIV, and the master key sits in the Android Keystore. On top of that, entry to the app can be locked with a passcode and biometrics.

Permissions the app requests:

Permission	What for
Internet and network state	talking to the panel
Notifications	local alerts
Biometrics	unlocking by fingerprint/face
Install packages	standard build only — to install an update; the F-Droid build doesn't have it

Worth remembering. A 3x-ui API token is full administrator access. The "Allow self-signed TLS" mode turns off certificate verification and is appropriate only for your own panel.

17. Panel version compatibility

The minimum panel is 3.3.0: that is where the panel starts accepting the API-token auth the whole app is built on. It will work against that version, but some capabilities arrived in later panel releases:

Capability	Panel needed
API-token sign-in, dashboard, inbounds, clients, subscriptions, Xray config	3.3.0
Outbound subscriptions	3.3.1
Nodes and mTLS, client IP log, client export and import, live inbound speed, online split by server	3.4.0
Bulk client actions, VLESS encryption key generation	3.4.1
Live per-client speed, outbound test, Target Strategy, route test, subscription announcement, MTProto and WireGuard client fields	3.5.0
SMTP sender settings, outbound-down alert threshold, "as the subscriber sees it" status, write-only node tokens, day-of-month traffic reset	3.6.0

The recommendation is simple: run the latest panel. Then every section of the app is available, and you get the panel's own fixes along the way.

If the panel is older than a section needs, the app **does not hide it** — it shows an explanation instead: your panel version doesn't support this, update the panel. That way you can see the feature exists and know what to do about it.

18. Troubleshooting

"Unexpected response" or a parsing error. The app received something that isn't JSON. Most often the panel address is missing its secret path (`webBasePath`), or a reverse proxy intercepts the request. Check the whole address by copying it from the browser.

A certificate error. Turn on "Allow self-signed TLS" in the connection settings — but only for your own panel.

"Your API token is no longer valid". The token was deleted or disabled in the panel. Create a new one and connect again; the saved profile fills in the rest.

A section says the panel doesn't support the feature. The panel is older than required — see section 17. Update it from the dashboard.

Xray changes have no effect. The configuration is saved in the panel but applied after an **Xray restart** — the button is on the dashboard.

No alerts arrive. Check that they are enabled in settings, that Android granted the notification permission, and that battery optimisation isn't killing the app. Remember that the check runs roughly every 30 minutes, and unreachability is reported after two consecutive failed checks.

The route test always says "default outbound". No inbound was picked — rules almost always key on it. For node inbounds the result isn't meaningful: the master panel has no routing rules for them.

No app update arrives. In the F-Droid build updates come only through the catalog and appear a day or two after a release — that is their build server's schedule, and it is out of our hands.

Created from an analysis of the app's source code. Yuriy Khachaturian (yukh.net)